

ChallengeHelper Class

Apex Code

```
public with sharing class ChallengeHelper {
    public static List<Challenge__c> queryChallenges(
        String status,
        Date startDate,
        Date endDate,
        String id,
        String name,
        String manager
    ) {
        String query = 'SELECT Id, Name, Status__c, Type__c, Description__c, Result__c,
SlackChannel__c, StartDate__c, EndDate__c, Manager__c, Manager__r.Name,
Participants__c, Reward__c FROM Challenge__c';
        List<String> conditions = new List<String>();

        // Build query conditions based on input
        if (status != null) {
            conditions.add('Status__c = :status');
        }
        if (startDate != null) {
            conditions.add('StartDate__c >= :startDate');
        }
        if (endDate != null) {
            conditions.add('EndDate__c <= :endDate');
        }
        if (id != null) {
            conditions.add('Id = :id');
        }
        if (name != null) {
            String likeName = '%' + name + '%';
            conditions.add('Name LIKE :likeName');
        }
        if (manager != null) {
            String likeManager = '%' + manager + '%';
            conditions.add('Manager__r.Name LIKE :likeManager');
        }

        // If no conditions, return empty results
        if (!conditions.isEmpty()) {
            query += ' WHERE ';
            query += String.join(conditions, ' AND ');
        }
        // Execute query and return results
        return Database.query(query);
    }

    public static List<Challenge__c> filterChallengesByParticipant(
```

```

        List<Challenge__c> challenges,
        String participantRole,
        String participantName
    ) {
        if (String.isBlank(participantRole) && String.isBlank(participantName)) {
            return challenges; // Return all challenges if no participant filter is
provided
        }

        String participantRoleFilter = participantRole != null ?
participantRole.toLowerCase().trim() : null;
        String participantNameFilter = participantName != null ?
participantName.toLowerCase().trim() : null;
        List<Challenge__c> filteredChallenges = new List<Challenge__c>();

        for (Challenge__c challenge : challenges) {
            if (
                challenge.Participants__c != null &&
                ((participantRoleFilter != null &&
challenge.Participants__c.toLowerCase().contains(participantRoleFilter)) ||
                (participantNameFilter != null &&
challenge.Participants__c.toLowerCase().contains(participantNameFilter)))
            ) {
                filteredChallenges.add(challenge);
            }
        }

        return filteredChallenges;
    }
}

```

Methods

queryChallenges(status, startDate, endDate, id, name, manager)

Signature

```

public static List<Challenge__c> queryChallenges(String status, Date startDate, Date
endDate, String id, String name, String manager)

```

Parameters

Name	Type	Description
status	String	
startDate	Date	
endDate	Date	
id	String	

name	String	
manager	String	

Return Type

List<Challenge__c>

filterChallengesByParticipant(challenges, participantRole, participantName)

Signature

```
public static List<Challenge__c> filterChallengesByParticipant(List<Challenge__c> challenges, String participantRole, String participantName)
```

Parameters

Name	Type	Description
challenges	List<Challenge__c>	
participantRole	String	
participantName	String	

Return Type

List<Challenge__c>